Quote displayLimitOrders(unsigned long lotSize, const std::vector<Order> &orders, const std::vector<Cancel> &cancels) {
# Implement this function
}

Given:

```
struct Quote final
{
  unsigned int bidPrice;
  unsigned int bidQuantity;
  unsigned int offerPrice;
  unsigned int offerQuantity;
}

struct Order final
{
  unsigned long timestamp;
  unsigned long orderID;
  unsigned int price;
  unsigned int quantity;
  unsigned int isBid;
}

struct Cancel final
{
  unsigned long orderID;
  unsigned long timestamp;
}
```

The STDIN:

```
10 #The lot size is 10
5 # There are 5 actions
0 1 10 50 B # at time 0, order #1 is issued as $10 for 50, bid
1 2 12 80 B # at time 1, order #2 is issued as $12 for 80, bid
2 3 150 50 B # at time 2, order #3 is issued as $150 for 50, bid
3 4 11 20 B # at time 3, order #4 is issued as $11 for 20, bid
2 4 CANCEL # at time 4, order # 2 is cancel.
```

Expected Output: 11 170 0 0

Where bids are B and offers are O.
The tracking system is tracking the sequence, higher prices for bids and lower prices for offers are considered "improvements". The best bid or offer should be reflected with the time.

```
#include <map>
#include <vector>
#include <algorithm>

Quote displayLimitOrders(unsigned long lotSize,
                         const std::vector<Order> &orders,
                         const std::vector<Cancel> &cancels) {
  // Event structure to merge orders and cancels
  struct Event {
    unsigned long timestamp;
    bool isCancel;
    size_t index;
  };

  // Merge all events
  std::vector<Event> events;
  for (size_t i = 0; i < orders.size(); i++) {
    events.push_back({orders[i].timestamp, false, i});
  }
  for (size_t i = 0; i < cancels.size(); i++) {
    events.push_back({cancels[i].timestamp, true, i});
  }

  // Sort by timestamp (timestamp in an ascending order)
  std::sort(events.begin(), events.end(), [](const Event& a, const Event& b) {
    return a.timestamp < b.timestamp;
  });

  // Order books: price -> total quantity at that price
  std::map<unsigned int, unsigned int> bids;
  std::map<unsigned int, unsigned int> offers;

  // Track active orders for cancellation
  std::map<unsigned long, Order> activeOrders;

  // Process events chronologically
  for (const auto& event : events) {
    if (!event.isCancel) {
      // Add order
      const Order& order = orders[event.index];
      activeOrders[order.orderID] = order;

      if (order.isBid) {
        bids[order.price] += order.quantity;
```

```
            } else {
                offers[order.price] += order.quantity;
            }
        } else {
            // Cancel order
            const Cancel& cancel = cancels[event.index];
            auto it = activeOrders.find(cancel.orderID);
            if (it != activeOrders.end()) {
                const Order& order = it->second;

                if (order.isBid) {
                    bids[order.price] -= order.quantity;
                    if (bids[order.price] == 0) {
                        bids.erase(order.price);
                    }
                } else {
                    offers[order.price] -= order.quantity;
                    if (offers[order.price] == 0) {
                        offers.erase(order.price);
                    }
                }
                activeOrders.erase(it);
            }
        }
    }

    Quote result = {0, 0, 0, 0};

    // Best bid = highest price
    if (!bids.empty()) {
        auto bestBid = bids.rbegin();
        result.bidPrice = bestBid->first;
        result.bidQuantity = bestBid->second;  // Return raw quantity
    }

    // Best offer = lowest price
    if (!offers.empty()) {
        auto bestOffer = offers.begin();
        result.offerPrice = bestOffer->first;
        result.offerQuantity = bestOffer->second;  // Return raw quantity
    }

    return result;
}
```